

Bound — Product Requirements Document

Working name: Bound

Tagline: Know your agent. Verify its mandate. Trust the transaction.

Hackathon: Yuno × Nauta NextWave 2026

Challenge: The Buyer Who Isn't Human

Version: 0.1

Primary integration path: AP2 → Bound Verify → Yuno Vault → Yuno Payments

1. Executive Summary

Bound is a trust and authorization layer for agentic commerce.

AI agents are increasingly able to discover products, select an offer and initiate a purchase, but payment infrastructure was designed around the assumption that the buyer pressing “Pay” is a human.

Bound allows a human to give an AI agent narrowly scoped, cryptographically verifiable purchasing authority without sharing raw payment credentials with that agent.

The system uses Google's Agent Payments Protocol (AP2) as its authorization model and Yuno as its payment infrastructure.

The core flow is:

Human

- creates an AP2-backed mandate
- delegates it to a verified agent
- agent discovers a purchase
- merchant presents a checkout
- Bound deterministically verifies the purchase
- Yuno retrieves the vaulted payment credential and executes the transaction
- Bound produces an auditable authorization trail.

Bound answers:

Is this exact agent authorized by this exact human to make this exact purchase right now?

Yuno answers:

How should this valid payment be securely processed?

2. Problem

Traditional payment systems primarily establish whether a payment credential can be charged.

Agentic commerce introduces additional questions.

A merchant receiving a purchase from an AI agent needs to know:

- Which agent is making the request?
- Who does the agent represent?
- Did that human authorize the agent?
- What exactly was it authorized to purchase?
- Is that authorization still active?
- Is this particular checkout within its limits?
- Has the authorization already been consumed?
- Was the mandate revoked?
- Who is accountable if the user later disputes the transaction?

Without those answers, merchants have two poor options:

Block automated buyers

This prevents legitimate agentic commerce.

Treat agents like ordinary users

This creates authorization ambiguity, fraud exposure and difficult disputes.

Bound creates the missing trust layer.

3. Why AP2

Bound does not invent a proprietary mandate format.

It adopts Google's Agent Payments Protocol as the authorization foundation.

AP2 provides a role-based architecture for secure agent purchases and defines Checkout Mandates and Payment Mandates. In autonomous flows, users can authorize open constraints in advance; an agent can later create a closed mandate for a specific purchase, provided that purchase satisfies the original constraints. Critical AP2 verification is designed to be deterministic.

Relevant AP2 primitives:

Trusted Surface

The non-agentic UI through which the human understands and authorizes the mandate.

Shopping Agent

The AI agent discovering and executing the purchase.

Open Checkout Mandate

The human's pre-authorized constraints over what may be purchased.

Closed Checkout Mandate

Authorization associated with one finalized merchant checkout.

Open Payment Mandate

The human's constraints over what may be paid and using which permitted credential.

Closed Payment Mandate

Authorization for the specific payment associated with the finalized checkout.

Checkout Receipt

Evidence of the merchant's acceptance or rejection of the checkout authorization.

Payment Receipt

Evidence of acceptance or rejection of the payment authorization.

Together, these artifacts create a verifiable chain from user intent through payment.

4. Why Yuno

Yuno is the payment execution layer.

Yuno's Agentic Commerce strategy already treats AI agents as emerging buyers and connects product discovery, purchasing and payments into existing merchant payment stacks. The company also explicitly emphasizes merchant control and visibility over AI-driven commerce.

Bound adds the authorization layer between agent intent and payment execution.

Yuno provides four important primitives for Bound.

4.1 Payment credential storage

Yuno can securely enroll a payment method and generate a persistent `vaulted_token` for future payments without repeatedly exposing the underlying payment details.

4.2 Payment execution

Yuno's Payments API supports creating payments using a previously stored `vaulted_token`.

4.3 Payment orchestration

Once Bound has authorized the transaction, Yuno remains responsible for routing and executing the payment through the merchant's configured payment infrastructure.

4.4 Agentic-commerce alignment

Yuno publicly describes agentic commerce as a new purchasing channel and has emphasized that autonomous payments require predefined limits, escalation paths and auditable control gates.

5. Product Thesis

Today:

```
graph TD; Agent --> Merchant; Merchant --> Payment;
```

The merchant has weak evidence about whether the human genuinely authorized the action.

With Bound:

```
graph TD; Human --> AP2[AP2 Mandate]; AP2 --> Agent; Agent --> Merchant; Merchant --> Bound[Bound Verify]; Bound --> Yuno; Yuno --> Payment;
```

Every payment has an explicit chain of authority.

6. Product Positioning

Bound is not:

- an AI shopping assistant;
- a wallet;
- a payment processor;
- another PSP;
- a replacement for Yuno;
- an LLM deciding whether transactions look acceptable.

Bound is:

authorization infrastructure for autonomous commerce.

The primary product is a merchant-facing verification layer accompanied by a consumer trusted surface for issuing and revoking mandates.

7. Primary Persona — Marta

Marta uses AI assistants frequently and wants them to perform purchases for her.

She wants the convenience of autonomous purchases but does not want to:

- share raw card details with agents;
- approve every transaction;
- give an agent unlimited spending authority;
- lose visibility into what was purchased;
- risk an old authorization remaining valid forever.

Her desired interaction is:

“Buy me this thing if these conditions become true.”

8. Secondary Persona — Merchant

VuelaYa is an online travel merchant.

It wants to accept purchases generated by AI agents because they represent a new revenue channel.

However, VuelaYa does not want to trust arbitrary bots.

For every agent-generated checkout it wants:

- verified delegation;
- verified authorization scope;

- revocation checks;
 - deterministic policy evaluation;
 - proof retained for later disputes.
-

9. Third Persona — Auditor / Payments Operations

An auditor or payment-operations analyst needs to answer:

Why was this payment accepted?

Bound provides a readable and cryptographically verifiable decision trail.

10. Primary User Journey

Step 1 — Add payment method

Marta adds her Visa card in Bound.

Bound uses the Yuno enrollment flow.

Yuno stores the credential and returns a `vaulted_token`.

Bound stores only an internal credential reference mapping to the Yuno token.

Example:

```
credential_crd_01
→ Yuno vaulted_token
→ Visa •••• 4242
```

The purchasing agent never receives the raw card details.

Step 2 — Register or connect an agent

Marta connects TravelBot.

Bound creates or registers:

```
agent_id
operator
```

```
public_key
status
created_at
```

Example:

```
TravelBot
agt_812

Operator
Bound

Identity
Verified

Public key
ES256 ...
```

Step 3 — Describe desired purchase

Marta says:

Buy me a flight from São Paulo to Córdoba for less than \$150 before September 15.
Economy only. Buy at most once.

The AI converts the request into structured constraints.

Important:

The AI may interpret the human's request.

The AI does not authorize the transaction.

The Trusted Surface shows the final deterministic conditions to Marta.

Step 4 — Human approves mandate

Marta sees:

```
TRAVEL TO CÓRDOBA

Agent
TravelBot

Route
```

São Paulo → Córdoba

Max amount
\$150

Cabin
Economy

Maximum purchases
1

Valid until
Sep 15

Payment method
Visa •••• 4242

She presses:

Authorize Agent

Bound creates and signs the AP2-backed open mandates.

Step 5 — Agent operates autonomously

TravelBot monitors merchant inventory.

No human is present.

Eventually it finds:

VuelaYa

GRU → COR
Sep 09

Economy

\$132

Step 6 — Merchant creates checkout

VuelaYa returns a merchant-signed checkout containing:


```
merchant
items
route
date
class
currency
total
checkout_id
```

The checkout becomes the specific object the agent wants to purchase.

Step 7 — Agent creates closed mandates

TravelBot binds its intended purchase to the merchant checkout.

It produces the AP2 artifacts required for verification.

Step 8 — Bound Verify

Bound receives:

```
agent
open mandate
closed mandate
merchant checkout
credential reference
```

Bound validates everything deterministically.

Possible results:

```
ALLOW
ESCALATE
DENY
```

Step 9 — Yuno payment

For `ALLOW`, Bound resolves the internal payment credential to its Yuno `vaulted_token`.

Bound's Yuno adapter creates the payment.

Conceptually:

```
POST /v1/payments
```

```
merchant_order_id  
amount  
customer  
payment_method:  
  type: CARD  
  vaulted_token: ...
```

Yuno processes the payment.

Step 10 — Receipt

Bound stores:

```
Checkout Mandate  
Checkout Receipt  
Payment Mandate  
Payment Receipt  
Yuno Payment ID  
decision  
policy evaluation  
timestamp
```

Marta receives a human-readable receipt.

11. Core Product Components

11.1 Bound Identity

Purpose:

Identify the purchasing agent.

Stores:

```
agent_id  
display_name  
operator  
public_key  
verification_status  
created_at  
revoked_at
```

Responsibilities:

- identify agent;
 - validate request signature;
 - prevent agent impersonation;
 - bind mandates to the intended agent public key.
-

11.2 Bound Trusted Surface

Purpose:

Safely obtain human authorization.

This is deliberately non-agentic.

Responsibilities:

- display interpreted conditions;
 - make monetary limits explicit;
 - show expiration;
 - show frequency;
 - show payment credential;
 - show assigned agent;
 - collect final consent;
 - initiate mandate signature;
 - never silently modify authorization.
-

11.3 Bound Mandates

Purpose:

Manage AP2 authorization artifacts.

States:

DRAFT
ACTIVE
REVOKED
EXPIRED
CONSUMED

A mandate may contain constraints such as:

merchant
category

```
items
maximum amount
currency
frequency
time window
destination
quantity
credential
agent
```

11.4 Bound Verify

Purpose:

Provide merchants with one deterministic authorization decision.

Core API:

```
POST /v1/verify
```

Input:

```
Agent Identity
AP2 open mandates
AP2 closed mandates
Merchant Checkout
```

Output:

```
{
  "decision": "ALLOW",
  "authorization_id": "auth_812",
  "reason": [
    "agent_signature_valid",
    "mandate_active",
    "checkout_matches_scope",
    "amount_within_limit",
    "usage_available"
  ]
}
```

No LLM participates in the final allow/deny computation.

11.5 Bound Revocation Registry

Purpose:

Make authorization revocation effective immediately.

Endpoint:

```
POST /v1/mandates/{id}/revoke
```

Every verification must read current mandate state.

Desired guarantee for hackathon:

```
revoke mandate
↓
next verification
↓
DENY
```

No manual deployment, UI refresh or operator action.

11.6 Bound Yuno Adapter

Purpose:

Bridge successful AP2 authorization into Yuno payment execution.

Responsibilities:

- resolve internal credential references;
- retrieve Yuno vaulted token;
- construct Yuno payment request;
- provide idempotency key;
- execute payment;
- normalize Yuno result;
- store Yuno payment ID;
- generate Bound/AP2 receipt information.

This bridge is one of the principal technical contributions of the project.

11.7 Bound Ledger

Purpose:

Store a tamper-evident audit trail.

Each authorization event records:

```
event_id
timestamp
actor
action
mandate_id
agent_id
merchant_id
checkout_hash
decision
reason
signature information
Yuno payment id
previous_event_hash
event_hash
```

For the hackathon, this can be an append-only hash chain.

It does not need to be a blockchain.

12. Authorization Engine

Authorization is a pure deterministic function.

Conceptually:

```
evaluate({
  agent,
  mandate,
  checkout,
  currentTime,
  previousUsage
})
```

Returns:

```
ALLOW | ESCALATE | DENY
```

Example policy:

```
Agent signature valid?
NO → DENY

Mandate active?
NO → DENY

Mandate expired?
YES → DENY

Merchant allowed?
NO → DENY

Category allowed?
NO → DENY

Amount <= autonomous threshold?
YES → continue

Amount <= human escalation threshold?
YES → ESCALATE

Otherwise
→ DENY

Frequency available?
NO → DENY

Replay detected?
YES → DENY

All rules satisfied
→ ALLOW
```

13. Human-in-the-Loop

Mandates can define three ranges.

Example:

```
0-150 USD
AUTO

150-200 USD
ASK
```

```
>200 USD  
DENY
```

A \$174 purchase returns:

```
ESCALATE
```

Marta receives:

```
TravelBot needs approval  
  
Flight to Córdoba  
$174  
  
Autonomous limit  
$150  
  
[ Approve once ]  
[ Increase mandate ]  
[ Reject ]
```

Approve once creates a narrowly scoped one-time authorization.

Example:

```
checkout = chk_812  
max amount = $174  
expires = +5 minutes  
uses = 1
```

It does not silently expand the original mandate.

14. Live Revocation

Revocation is a first-class requirement.

Example:

```
14:03:10  
Mandate ACTIVE  
  
14:03:11  
Marta presses REVOKE
```



```
14:03:11
Mandate REVOKED
```

```
14:03:12
TravelBot submits checkout
```

```
14:03:12
DENY
reason = mandate_revoked
```

The demo must make this latency visibly obvious.

15. Replay Protection

Every agent request contains:

```
nonce
timestamp
agent signature
mandate id
checkout hash
```

Once an authorization is consumed, replaying it fails.

Example:

```
REQUEST 1
ALLOW

REQUEST 2
same nonce

DENY
REPLAY_DETECTED
```

16. Frequency and Aggregate Limits

The system must prevent agents from bypassing a limit by splitting purchases.

Mandate:

```
Maximum total
$150
```

Attack:

```
$100
$100
$100
```

The first may succeed.

Subsequent purchase must calculate accumulated usage.

```
spent = $100
remaining = $50

requested = $100

DENY
aggregate_limit_exceeded
```

17. Rich Conditions

Hackathon MVP should support a constrained set.

Required:

```
maximum amount
currency
merchant
category
expiration
maximum uses
```

Demo-specific:

```
origin
destination
travel class
```

Stretch:

```
price condition
time of day
weekly/monthly aggregate limit
quantity
SKU
subscription cadence
```

18. Adversarial Agent Protection

The judges should be able to attack the agent.

Bound should survive:

Agent impersonation

Different private key attempts to use TravelBot's mandate.

Result:

```
DENY
invalid_agent_signature
```

Expired mandate

Result:

```
DENY
mandate_expired
```

Revoked mandate

Result:

```
DENY
mandate_revoked
```

Excess price

Result:

DENY
amount_exceeded

Unauthorized merchant/category

Result:

DENY
scope_violation

Split payment attack

Result:

DENY
aggregate_limit_exceeded

Replay

Result:

DENY
replay_detected

Changed checkout

Checkout details changed after signature/hash.

Result:

DENY
checkout_integrity_failure

19. Dispute Resolution

Marta later clicks:

I don't recognize this purchase.

Bound reconstructs the authorization chain.

Example:

Purchase

\$132

Agent

TravelBot

✓ signature valid

Human delegation

Marta → TravelBot

✓

Mandate

Flight to Córdoba

≤ \$150

✓

Merchant checkout

GRU → COR

\$132

✓

Mandate state at transaction

ACTIVE

✓

Checkout integrity

✓

Payment authorization

✓

Yuno payment

SUCCEEDED

✓

Bound can then show:

AUTHORIZATION EVIDENCE

The transaction was inside
the active mandate.

AP2 explicitly designs Checkout/Payment Mandates and corresponding Receipts so they can later be assembled as evidence in disputes.

20. User Interfaces

Screen A — Home

BOUND

Your agents can spend.
Only on your terms.

ACTIVE AGENTS

TravelBot
2 mandates
\$132 spent this month

ACTIVE MANDATES

Córdoba Flight
\$150 max
1 purchase
ACTIVE

Screen B — Agent Passport

TravelBot

Verified Agent

Operator
Bound

Agent ID
agt_812

Identity
Verified

Public key
Verified

Active mandates
2

Successful purchases
14

Rejected attempts
3

Screen C — Create Mandate

Primary interface starts with natural language:

What should your agent be allowed to do?

"Buy me a flight to Córdoba
if it costs less than \$150."

AI proposes structured policy.

Human reviews deterministic representation.

Screen D — Mandate Detail

Córdoba Flight

ACTIVE

Agent
TravelBot

Route
São Paulo → Córdoba

Limit
\$150

Cabin
Economy

Uses
0 / 1

Expires
September 15

Visa •••• 4242

[REVOKE MANDATE]

Screen E — Merchant Verification

Merchant-facing visualization:

AGENT PURCHASE REQUEST

TravelBot
Verified

Authorization
PASS

Agent Identity	✓
Mandate	✓
Checkout integrity	✓
Amount	✓
Scope	✓
Expiration	✓
Revocation	✓

AUTHORIZED

[Execute via Yuno]

Screen F — Audit Timeline

14:03:01
Merchant checkout signed

14:03:02
Agent signature verified

14:03:02
Mandate verified

14:03:02
Policy evaluation
ALLOW

14:03:03
Yuno payment created

14:03:04
Payment succeeded

14:03:04
Receipt sealed

21. Merchant SDK Vision

Future interface:

```
const decision = await bound.verify({
  checkout,
  agentProof,
  mandates,
});

if (!decision.allowed) {
  return reject(decision);
}

const payment = await bound.pay({
  authorizationId: decision.authorizationId,
});
```

Or:

```
const result = await bound.authorizeAndPay({
  checkout,
  agentProof,
  mandates,
});
```

The merchant should not need to understand AP2 internals.

22. Proposed Backend API

```
POST /agents
GET  /agents/:id

POST /credentials
GET  /credentials

POST /mandates
GET  /mandates/:id
POST /mandates/:id/revoke
```

```
POST /merchant/checkouts
POST /verify

POST /authorizations/:id/pay

GET /transactions/:id
GET /transactions/:id/audit

POST /transactions/:id/dispute
```

23. Core Data Model

User

```
id
name
email
signing_key_reference
created_at
```

Agent

```
id
name
operator
public_key
status
created_at
```

Credential

```
id
user_id
provider = YUNO
vaulted_token_encrypted
display_brand
last_four
status
```

Mandate

```
id
user_id
agent_id
credential_id
status
valid_from
valid_until
max_amount
currency
max_uses
current_uses
aggregate_spend
constraints_json
ap2_open_checkout_mandate
ap2_open_payment_mandate
created_at
revoked_at
```

Checkout

```
id
merchant_id
signed_checkout_jwt
checkout_hash
amount
currency
items
metadata
```

Authorization

```
id
mandate_id
checkout_id
agent_id
decision
reasons
nonce
created_at
consumed_at
```

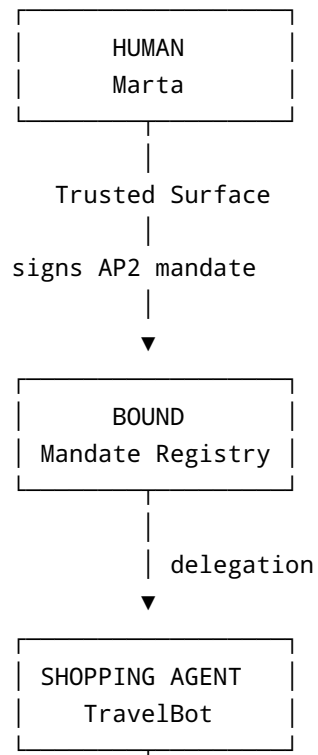
Payment

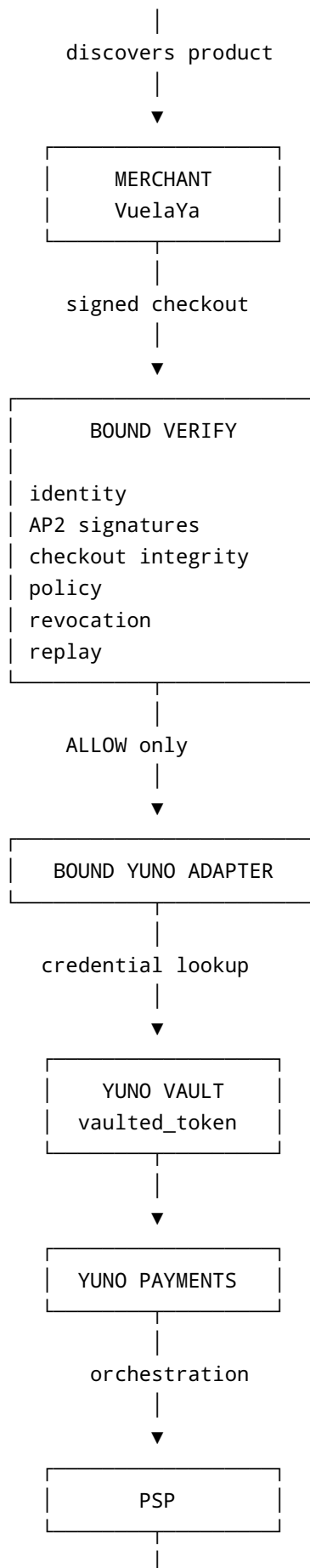
```
id
authorization_id
yuno_payment_id
status
amount
currency
created_at
```

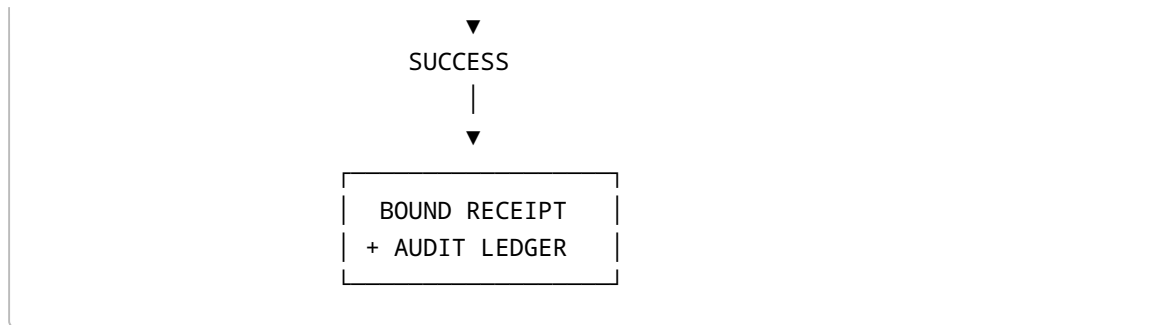
AuditEvent

```
id
transaction_id
type
actor
payload_hash
previous_hash
event_hash
created_at
```

24. Architecture







25. Security Principles

Never trust the agent

The agent is considered potentially adversarial.

Never let the LLM authorize money

LLM:

natural language
→ structured policy proposal

Deterministic engine:

structured policy + checkout
→ ALLOW / ESCALATE / DENY

Least authority

Mandates should contain the smallest practical:

- duration;
- amount;
- scope;
- number of uses.

Payment credential isolation

The agent should receive neither:

- PAN/card number;
- CVV;
- raw Yuno vaulted token.

It receives only a logical credential reference or proof.

Fail closed

If Bound cannot determine that a purchase is authorized:

DENY

or:

ESCALATE

Never:

probably okay → charge

26. Idempotency

Payment execution must be idempotent.

Bound creates:

```
idempotency_key =  
authorization_id
```

Multiple retries against the same authorization must not cause multiple Yuno charges.

Yuno's payment/enrollment APIs use idempotency keys in their documented integration flows.

27. Hackathon MVP

The MVP must optimize for the judge's trial by fire.

It will implement one vertical:

Autonomous flight purchasing.

Required working capabilities:

1. Enroll Yuno payment method
2. Register TravelBot
3. Create mandate
4. Sign/represent AP2 authorization
5. Agent discovers fictional flight
6. Merchant creates checkout
7. Bound verifies
8. Yuno Sandbox payment executes
9. Audit trail renders
10. Outside-limit purchase fails
11. Live revocation fails immediately
12. Impersonated agent fails

28. What We Will Fake

Allowed/mockable:

- airline inventory;
- merchant product discovery;
- flight prices;
- merchant brand;
- waiting for price changes;
- transaction volume.

Should be real:

- agent execution;
- mandate state;
- deterministic verification;
- signature verification;
- revocation;
- Yuno integration where sandbox credentials permit;
- authorization decisions;
- audit trail.

29. What We Will NOT Build

For the hackathon:

- real airline integration;
- full AP2 ecosystem implementation;
- complete PCI infrastructure;
- x402;
- cryptocurrency;
- decentralized identity;

- production KYC;
- real card-network dispute submission;
- generalized constraint DSL;
- blockchain;
- marketplace;
- custom payment processor.

These dilute the core story.

30. Trial-by-Fire Test Matrix

Test A — normal purchase

Mandate:

≤ \$150
Córdoba
economy

Purchase:

\$132
Córdoba
economy

Expected:

ALLOW

Test B — amount exceeded

Purchase:

\$300

Expected:

DENY
amount_exceeded

Test C — wrong category

Purchase:

hotel \$120

Expected:

DENY
scope_violation

Test D — expired mandate

Judge changes clock/expiration.

Expected:

DENY
mandate_expired

Test E — live revocation

Judge presses revoke.

Next attempt:

DENY
mandate_revoked

Test F — impersonated agent

Request signed with different key.

Expected:

DENY
invalid_agent_signature

Test G — modified checkout

Price changes after checkout signature.

Expected:

DENY
checkout_integrity_failure

Test H — replay

Previously accepted authorization repeated.

Expected:

DENY
authorization_consumed

31. Demo Sequence

Act 1 — The problem

Show TravelBot ready to purchase.

Explain:

Existing payment infrastructure knows whether a card can pay. It doesn't know whether an AI agent should.

Act 2 — Give authority

Marta:

Buy a flight to Córdoba under \$150.

Show the structured mandate.

Click:

Authorize TravelBot

Act 3 — Successful autonomous purchase

TravelBot finds:

\$132 flight

Merchant sees verification checks passing.

Yuno payment succeeds.

Marta receives record.

Act 4 — Agent crosses boundary

TravelBot attempts:

\$300 flight

Immediately:

DENIED

Explain:

The LLM never decides whether money moves.

Act 5 — Trial by fire

Hand control to judge.

Judge presses:

Revoke mandate

TravelBot tries the previously valid \$132 purchase.

Result:

DENIED
Mandate revoked 713 ms ago

This should be the strongest demo moment.

Act 6 — Audit

Open transaction.

Show:

Human
Agent
Mandate
Merchant checkout
Verification
Yuno payment
Receipt

End with:

Every transaction tells us who authorized whom, to buy what, under which limits, and exactly why the payment was accepted.

32. Success Metrics

Hackathon success

Primary:

100% deterministic pass
of trial-by-fire cases

Secondary:

<1 second
revocation → enforcement

0 raw card exposure
to agent

100% purchase decisions
have readable audit trail

33. Product Metrics — Future

If commercialized:

Agent payment acceptance rate

Percentage of agent-originated legitimate transactions merchants safely accept.

Authorization rejection accuracy

Percentage of unauthorized agent actions correctly blocked.

False rejection rate

Legitimate agent purchases incorrectly blocked.

Human escalation rate

Percentage requiring explicit human approval.

Dispute evidence coverage

Percentage of transactions containing a complete mandate/receipt chain.

Merchant integration time

Time required to become agent-payment-ready.

34. Future Product Expansion

Bound Agent Passport

Portable Know Your Agent identity.

Bound SDK

One-call merchant verification.

Bound Policy Language

Reusable programmable spending conditions.

Bound Reputation

History of compliant/failed agent interactions.

Bound Disputes

Automated assembly of mandate evidence.

Bound AP2/Yuno Adapter

Open-source integration library connecting AP2 authorization objects to Yuno payment execution.

Bound Enterprise

Corporate procurement mandates:

```
Restock SKU 193
when stock < 100

≤ 500 units

≤ $7.20/unit

supplier ∈ approved list
```

35. Long-Term Vision

Humans currently give software credentials.

In an agentic world, humans need to give software **authority**.

Authority must be:

```
specific
limited
revocable
verifiable
auditable
portable
```

Bound aims to become the authorization layer for that world.

Payment rails may change.

Agents may change.

Merchants may change.

The underlying question remains:

Did this human authorize this agent to perform this exact economic action?

36. Core Pitch

Every payment system answers one question:

Can this payment method pay?

But when an AI agent is the buyer, there's a more important question:

Should this agent be allowed to pay?

We built Bound.

Bound uses Google's AP2 to turn a human's intent into verifiable purchasing authority.

The agent never receives the raw card.

Bound verifies the identity, mandate, checkout, limits and revocation state in deterministic code.

Only after authorization passes does Yuno retrieve the stored credential and execute the payment.

Yuno makes agents able to pay.

Bound makes them safe to trust.

37. Reference Architecture Status

The components are grounded in existing technology, but the complete product is new.

Existing

Google AP2: authorization roles, mandates, signatures, constraints, receipts and dispute evidence.

Yuno: Agentic Commerce, payment orchestration, secure payment-method enrollment and reusable vaulted tokens.

Yuno Payments: payment creation using a stored `vaulted_token`.

Our contribution

AP2 Mandates



Bound deterministic verification



credential scoping / revocation



Yuno Vault adapter



Yuno payment execution



AP2-style authorization evidence

There is no assumption in this PRD that Yuno currently ships a native AP2 adapter.

Building that bridge is the central technical thesis of Bound.